

Le langage procédural en SQL sous PostGres

Les triggers

PL/pgSQL

(Procedural Language/postgreSQL Structured Query Language)

Les triggers -Procédures déclenchées-

■ Introduction

- Les triggers peuvent être comparé aux règles (rule) avec tout de même un nuance de taille
 - Les règles ne déclenche que des actions des types requêtes SQL (update/delete/insert/select)
 - Alors que les triggers peuvent déclencher des traitements algorithmiques sur les données
- Les triggers, comme les règles se déclenchent sur les événements update, delete, insert
 - Contrairement aux règles qui se déclenchent sur l'un de ces 3 événements, les triggers peuvent se déclencher sur 1, 2 ou les trois événements
- Les triggers comme les règles sont associés à une table uniquement, pas à une vue, contrairement aux règles

Les triggers -Procédures déclenchées-

■ Définition

- Un trigger est déclenché avant ou après un INSERT, un UPDATE ou un DELETE.
- Un trigger utilise une fonction trigger, ce sont des fonctions particulières sans paramètre et de type trigger
- Lors de l'exécution d'un trigger, des variables spéciales sont créées automatiquement : NEW, OLD, ...
- Après la définition de la fonction, il faut encore définir le trigger lui-même avec CREATE TRIGGER ...

Attention il faut d'abord définir la fonction trigger avant le trigger

Les triggers -Procédures déclenchées-

■ Création d'une fonction trigger

□ Syntaxe :

```
CREATE FUNCTION nomfonc() RETURNS trigger AS $$  
BEGIN  
    ...  
    RETURN NEW;  
END; $$ LANGUAGE plpgsql;
```

- Une fonction trigger doit renvoyer soit NULL soit une valeur record ayant exactement la même structure que la table sur laquelle le trigger a été lancé.

Les triggers -Procédures déclenchées-

■ Création d'un trigger

■ Syntaxe

```
CREATE TRIGGER nom { BEFORE | AFTER } { evenement [ OR ... ] }  
ON table [ FOR [ EACH ] { ROW | STATEMENT } ]  
EXECUTE PROCEDURE nomfonc ()
```

nom : Le nom du nouveau déclencheur.

BEFORE | AFTER : Détermine si la fonction est appelée avant ou après l'événement.

evenement : événement déclencheur (INSERT, UPDATE ou DELETE);

table : nom de la table à laquelle est rattaché le déclencheur.

FOR EACH ROW | FOR EACH STATEMENT : Précise si la procédure du déclencheur doit être lancée pour chaque ligne affectée par l'événement ou simplement pour chaque instruction SQL. FOR EACH STATEMENT est la valeur par défaut.

nomfonc : nom de la fonction utilisateur, déclarée sans argument et renvoyant le type trigger, exécutée à l'activation du déclencheur.

Les triggers -Procédures déclenchées-

■ Création d'une fonction trigger

- Quand une fonction PL/pgSQL est appelée en tant que trigger, plusieurs variables spéciales sont créées :

NEW : variable contenant la nouvelle ligne pour les opérations INSERT/UPDATE dans les triggers de niveau ligne (*FOR EACH ROW*). Cette variable est NULL dans un trigger de niveau instruction (*FOR EACH STATEMENT*)

OLD : variable contenant la ligne affectée pour les opérations UPDATE/DELETE dans les triggers de niveau ligne. Cette variable est NULL dans les triggers de niveau instruction (*FOR EACH STATEMENT*)

TG_NAME : nom du trigger réellement lancé.

TG_WHEN : BEFORE soit AFTER, selon la définition du trigger.

TG_LEVEL : ROW soit STATEMENT, selon la définition du trigger.

TG_OP : INSERT, UPDATE ou DELETE, indiquant pour quelle opération le trigger a été lancé.

TG_RELID : identifiant de l'objet ayant déclenché le trigger.

TG_TABLE_NAME : nom de la table qui a déclenché le trigger.

TG_TABLE_SCHEMA : nom du schéma de la table qui a appelé le trigger.

Les triggers -Procédures déclenchées-

■ Création d'une fonction trigger

□ Les structures OLD et NEW:

- Les structures **OLD** et **NEW** sont générées automatiquement lors du déclenchement du trigger en suivant les règles suivantes :
 - Les triggers de niveau ligne (FOR EACH ROW) lancés BEFORE peuvent renvoyer NULL pour indiquer au gestionnaire de trigger de ne pas exécuter l'instruction SQL (INSERT/UPDATE/DELETE) qui a déclenché le trigger pour cette ligne.
 - Si une valeur non NULL est renvoyée alors l'instruction SQL se déroule avec cette valeur ligne. Renvoyer une valeur ligne différente de la valeur originale de NEW modifie la ligne qui sera insérée ou mise à jour (mais n'a pas d'effet sur le cas DELETE).
 - Pour modifier la ligne à stocker, il est possible de remplacer des valeurs seules directement dans NEW et de renvoyer NEW, ou de construire un nouveau record/ligne à renvoyer.
 - Les triggers de niveau instruction (FOR EACH STATEMENT) BEFORE ou AFTER ou un trigger de niveau ligne AFTER ignore toujours la valeur de retour ;

Les triggers -Procédures déclenchées-

■ Exemple

□ Définition de la fonction trigger

```
CREATE or replace FUNCTION emp_stamp() RETURNS trigger AS $$  
BEGIN -- Verifie que nom_employe et salaire sont donnés  
    IF NEW.nom_employe IS NULL  
        THEN RAISE EXCEPTION ' 1''employe doit avoir un nom';  
    END IF;  
    IF NEW.salaire IS NULL  
        THEN RAISE EXCEPTION '% doit avoir un salaire', NEW.nom_employe;  
    END IF;  
    -- Qui travaille pour ce salaire ?  
    IF NEW.salaire < 0 THEN  
        RAISE EXCEPTION '% doit avoir un salaire positif', NEW.nom_employe;  
    END IF; -- qui a changé le salaire et quand  
    NEW.date_dermodif := current_timestamp;  
    NEW.utilisateur_dermodif := current_user;  
    RETURN NEW;  
END; $$ LANGUAGE plpgsql;
```

Colonne	Type	NOT NULL
nom_employe	character varying(25)	NOT NULL
salaire	integer	NOT NULL
date_dermodif	date	
utilisateur_dermodif	character varying(25)	

Les triggers -Procédures déclenchées-

■ Exemple

□ Définition du trigger

```
CREATE TRIGGER emp_stamp BEFORE INSERT OR UPDATE ON emp FOR  
EACH ROW EXECUTE PROCEDURE emp_stamp();
```

□ Test du déclenchement du trigger

```
INSERT INTO emp VALUES ('Carmignac', -4000, null, null);  
INSERT INTO emp VALUES ('Carmignac', 4000, null, null);  
-- insertion réalisée avec succès
```

```
UPDATE emp SET salaire = -3000
```

```
WHERE nom_employe LIKE 'Carmignac'; -- provoque une erreur
```

Erreur SQL :

ERROR: Carmignac doit avoir un salaire positif

Dans l'instruction :

insert into emp values('Carmignac',-4000,null,null);

nom_employe	salaire	date_dermodif	utilisateur_dermodif
Carmignac	4000	2020-05-15	carmigna

Les triggers -Procédures déclenchées-

■ Autre exemple :

■ tracer les modifications sur une table

- Nous allons tracer dans une table log_formation les événements survenues sur la table formation, cad savoir si il a eu une insertion, modification ou suppression, a quelle date et heure et qui est l'auteur de l'événement

La table log :

```
CREATE TABLE log_formation (  
    id_formation int,  
    intitule_form TEXT,  
    nbheures int default '0',  
    niveau varchar(50) default NULL,  
    Even char(1),  
    date_even timestamp,  
    user_even text ) ;
```

Colonne	Type
id_formation	integer
intitule_formation	text
nbheures	integer
niveau	character varying(50)

Les triggers -Procédures déclenchées-

■ Autre exemple :

■ tracer les modifications sur une table

```
CREATE or replace FUNCTION log_formation() RETURNS trigger AS $$
BEGIN -- Verifie que nom_employe et salary sont donnés
  IF (TG_OP = 'DELETE') THEN
    INSERT INTO log_formation SELECT OLD.*, 'D', current_timestamp, user;
  ELSIF (TG_OP = 'UPDATE') THEN
    INSERT INTO log_formation SELECT NEW.*, 'U', current_timestamp, user;
  THEN
    INSERT INTO log_formation SELECT NEW.*, 'I', current_timestamp, user;
  END IF;
END IF;

RETURN NULL;
END; $$ LANGUAGE plpgsql;
```

Les triggers -Procédures déclenchées-

- Autre Exemple :
 - tracer les modifications sur une table

- Définition du trigger

```
CREATE TRIGGER trigger_log_formation AFTER INSERT OR DELETE OR UPDATE ON  
formation FOR EACH ROW EXECUTE PROCEDURE log_formation();
```

- Test du déclenchement du trigger

```
INSERT INTO formation values (50,'Formation pl/pgSQL',45,'Confirme');  
UPDATE formation SET nbheures=55 WHERE id_formation=50;  
DELETE FROM formation WHERE id_formation=50;
```

id_formation	intitule_form	nbheures	niveau	even	date_even	user_even
50	Formation pl/pgSQL	45	Confirme	I	2011-11-17 10:50:30.179075	carmignac.l
50	Formation pl/pgSQL	55	Confirme	U	2011-11-17 10:50:30.179075	carmignac.l
50	Formation pl/pgSQL	55	Confirme	D	2011-11-17 10:50:30.179075	carmignac.l



FIN